

What is OOP?

Object Oriented Programming is software design method that models the characteristics of abstract or real objects using classes and objects.

Basic Concepts of Object Oriented Programming

1. Object and Class

Objects are important runtime entities in object oriented method.

They may characterize a location, a bank account, and a table of data or any entry that the program must handle.

Each object holds data and code to operate the data. Object can interact without having to identify the details of each other's data or code.

For Example: Student, Person, Car

A class is a set of objects with similar properties (attributes), common behaviour (operations), and common link to other objects.

The objects are variable of type class. A class is a collection of objects of similar type. Classes are user defined data types and work like the built-in type of the programming language.

2. Data Abstraction and Encapsulation

Data abstraction refers to the act of representing important description without including the background details or explanations.

Classes use the concept of abstraction and are defined as a list of abstract attributes such as size, cost and functions operate on these attributes. They summarize all the important properties of the objects that are to be created.

Classes use the concepts of data abstraction and it is called as Abstract Data Type (ADT).

Data Encapsulation means wrapping of data and functions into a single unit (i.e. class). It is most useful feature of class. The data is not easy to get to the outside world and only those functions which are enclosed in the class can access it.

These functions provide the boundary between Object's data and program. This insulation of data from direct access by the program is called as **Data hiding**.

3. Inheritance

Inheritance is the process by which objects of one class can get the properties of objects of another class.

Inheritance means one class of objects inherits the data and behaviours from another class. Inheritance maintains the hierarchical classification in which a class inherits from its parents.

4. Polymorphism

(Poly means —many and morph means —form).

Polymorphism means the ability to take more than one form.

Polymorphism plays a main role in allocate objects having different internal structures to share the same external interface.

5. Dynamic Binding

Binding refers to the linking of a procedure call to the code to be executed in response to the call. Dynamic binding means that the code related with a given procedure call is not known until the time of the call at run time.

Dynamic binding is associated polymorphism and inheritance.

6. Message Passing

Objects communicate with one another by sending and receiving information.

A message for an object is a request for execution of procedure, and therefore will invoke the function.

Benefits and Applications of OOP

1. Real time systems
2. Simulation and modelling
3. Object Oriented Database
4. Hypertext and Hypermedia
5. AI and Expert System
6. Neural network and Parallel programming
7. Decision support and office automation system.

Basics of JAVA

History of JAVA

Java is a general-purpose, object-oriented programming language developed by Sun Microsystems of USA in 1991.

Originally called Oak by James Gosling (one of the inventor of the language).

Java was invented for the development of software for consumer electronic devices like TVs, toasters, etc. The main aim had to make java simple, portable and reliable.

Java Features

1. Compiled and Interpreted
2. Platform Independent and portable
3. Object- oriented
4. Robust and secure
5. Distributed
6. Familiar, simple and small
7. Multithreaded and Interactive
8. High performance
9. Dynamic and Extensible

1. Compiled and Interpreted: Basically a computer language is either compiled or interpreted. Java comes together both these approach thus making Java a two-stage system.

Java compiler translates Java code to Bytecode instructions and Java Interpreter generate machine code that can be directly executed by machine that is running the Java program.

2. Platform Independent and portable: Java supports the feature portability. Java programs can be easily moved from one computer system to another and anywhere. Changes and upgrades in operating systems, processors and system resources will not force any alteration in Java programs.

Java certifies portability in two ways.

First way is, Java compiler generates the bytecode and that can be executed on any machine. Second way is, size of primitive data types are machine independent.

3. Object-oriented: Java is truly object-oriented language. In Java, almost everything is an Object. All program code and data exist in objects and classes. Java comes with an extensive set of classes; organize in packages that can be used in program by Inheritance.

4. Robust and secure: Java is a most strong language which provides many securities to make certain reliable code. It is design as garbage –collected language, which helps the programmers virtually from all memory management problems. Java also includes the concept of exception handling, which detain serious errors and reduces all kind of threat of crashing the system.

Security is an important feature of Java and this is the strong reason that programmer use this language for programming on Internet.

5. Distributed: Java is called as Distributed language for construct applications on networks which can contribute both data and programs. Java applications can open and access remote objects onInternet easily. That means multiple programmers at multiple remote locations to work together onsingle task.

6. Simple and small: Java is very small and simple language. Java does not use pointer and header files, goto statements, etc. It eliminates operator overloading and multiple inheritance.

7. Multithreaded and Interactive: Multithreaded means managing multiple tasks simultaneously. Java maintains multithreaded programs. That means we need not wait for the application to complete one task before starting next task. This feature is helpful for graphic applications.

8. High performance: Java performance is very extraordinary for an interpreted language, majorly due to the use of intermediate bytecode. Java architecture is also designed to reduce overheads during runtime. The incorporation of multithreading improves the execution speed of program.

9. Dynamic and Extensible: Java is also dynamic language. Java is capable of dynamically linking in new class, libraries, methods and objects. Java can also establish the type of class through the query building it possible to either dynamically link or abort the program, depending on the reply.

Java program supports functions written in other language such as C and C++, known as **native methods**.

Java Development kit (JDK) – The JDK comes with a set of tools that are used for developing and running Java program.

It includes:

1. Appletviewer(It is used for viewing the applet)
2. Javac(It is a Java Compiler)
3. Java(It is a java interpreter)
4. Javap(Java diassembler,which convert byte code into program description)
5. Javah(It is for java C header files)
6. Javadoc(It is for creating HTML document)
7. Jdb(It is Java debugger)

For compiling and running the program we have to use following commands:

a) javac (Java compiler)

In java, we can use any text editor for writing program and then save that program with —.java extension. Java compiler converts the source code or program in bytecode and interpreter convert —.java file in —.class file.

Syntax:

C:\javac filename.java

If my filename is —abc.java then the syntax will be

C:\javac abc.java

b) java(Java Interpreter)

As we learn that, we can use any text editor for writing program and then save that program with —.java extension. Java compiler converts the source code or program in bytecode and interpreter convert —.java file in —.class file.

Syntax:

C:\java filename

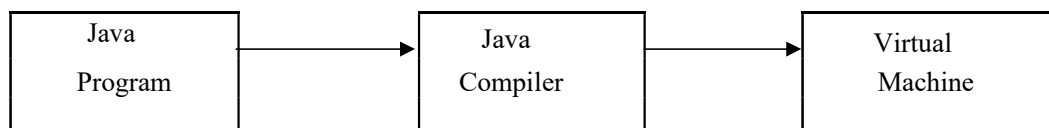
If my filename is abc.java then the syntax will be

C:\java abc

Comparison in Java and C++

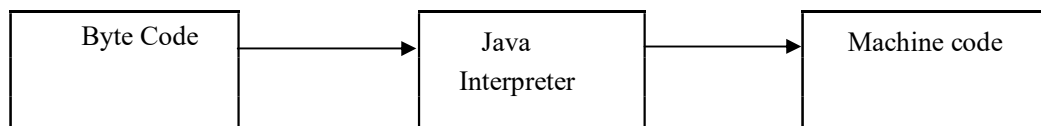
	Java	C++
1	Java is true Object-oriented language.	C++ is basically C with Object-oriented extension.
2	Java does not support operator overloading.	C++ supports operator overloading.
3	It supports labels with loops and statement blocks	It supports goto statement.
4	Java does not have template classes as in C++.	C++ has template classes.
5	Java compiled into byte code for the Java Virtual Machine. The source code is independent on operating system.	Source code can be written to be platform independent and written to take advantage of platform.C++ typically compiled into machine code.
6	Java does not support multiple inheritance of classes but it supports interface.	C++ supports multiple inheritance of classes.
7	Runs in a protected virtual machine.	Exposes low-level system facilities.
8	Java does not support global variable. Every variable should declare in class.	C++ support global variable.
9	Java does not use pointer.	C++ uses pointer.
10	It Strictly enforces an object oriented programming paradigm.	It Allows both procedural programming and object-oriented programming.
11	There are no header files in Java.	We have to use header file in C++.

Java Virtual machine(JVM) and Byte Code: As we know that all programming language compilers convert the source code to machine code.Same job done by Java Compiler to run a Java program, but the difference is that Java compiler convert the source code into Intermediate code is called as **bytecode**. This machine is called the *Java Virtual machine* and it exists only inside the computer memory.



Source Code

Byte Code

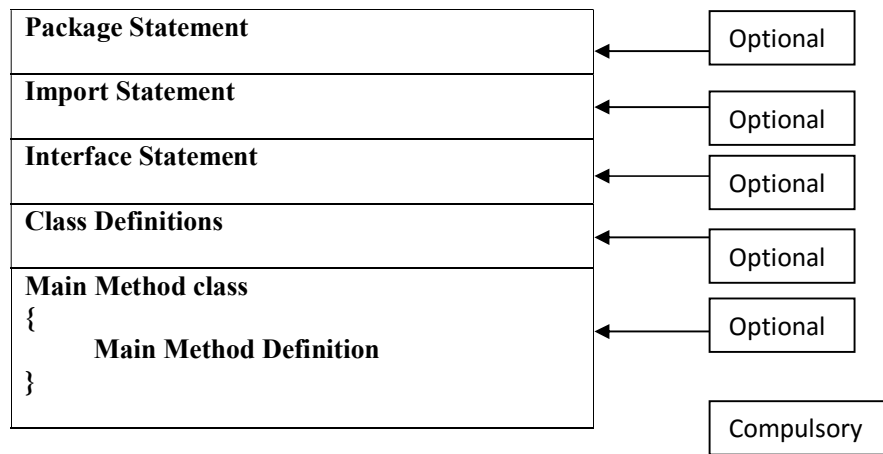


Virtual machine

Real Machine

Java Program Structure

Documentation Section



1. **Documentation Section:** It comprises a set of Comment lines giving the name, the author and other details of program, which the programmer would like to refer to at a later stage.
2. **Package Statement:** It declares a package name and informs a compiler that the classes defined here belong to this package.

```
package student;
```
3. **Import Statement:** It instructs the interpreter to load the classes contained in specific packages

```
import student.test;
```
4. **Interface Statement:** An interface is like a class but includes a group of method declarations.
5. **Class Definitions:** A Java program may contain multiple class definitions which can be defined in this section.

6. **Main Method Class:** Main method is the starting point of a Java program and so there must be a class with main method in every Java program.

A main method creates objects of various classes and establishes communication between them.

```
class FirstProgram
{
    public static void main(String args[])
    {
        System.out.println("—This is my first program");
    }
}
```

Here,

z-public is an access specifier that declares the main method as unprotected and therefore making it accessible to all other classes.

-static is keyword which can be called & executed without creating the object, so when we want to call main() method without using an object we should declare a main() method as static.

-void means no value & as main() method doesn't return any value, its return type is set to void.

-String args[] is array of String type which is used to store command line arguments.

Building Blocks

Tokens

There are five types of token as follows:

1. Literals
2. Identifiers
3. Keyword
4. Operators
5. Separators

- 1. Literals:** Literals in Java are a sequence of characters (digits, letters and other characters) that characterize constant values to be stored in variables. Java language specifies five major types of literals are as follows:

1. Integer literals
2. Floating point literals
3. Character literals
4. String literals
5. Boolean literals

- 2. Identifiers:** Identifiers are programmer-created tokens. They are used for naming classes, methods, variables, objects, labels, packages and interfaces in a program. Java identifiers follow the following rules:

1. They can have alphabets, digits, and the underscore and dollar sign characters.
2. They must not start with a digit.
3. Uppercase and lowercase letters are individual.
4. They can be of any length.

Identifier must be meaningful, easily understandable and descriptive.

For example: Private and local variables like `—lengthl`. Name of public methods and instance variables

- 3. Keywords:** Keywords are important part of Java. Java language has reserved 50 words as keywords. Keywords have specific meaning in Java. We cannot use them as variable, classes and method. Following table shows keywords.

abstract	char	catch	Boolean
----------	------	-------	---------

default	finally	do	implements
If	long	throw	Private
package	static	break	Double
This	volatile	import	protected
class	throws	byte	Else
float	final	public	Transient
native	instanceof	case	Extends
Int	null	const	New
return	try	for	Switch
interface	void	while	synchronized
short	continue	goto	Super
assert	const		

- 4. Operator:** Java carries a broad range of operators. An operator is symbols that specify operation to be performed may be certain mathematical and logical operation. Operators are used in programs to operate data and variables. They frequently form a part of mathematical or logical expressions. Categories of operators are as follows:

1. Arithmetic operators
2. Logical operators
3. Relational operators
4. Assignment operators
5. Conditional operators
6. Increment and decrement operators
7. Bit wise operators

4.1 Arithmetic operators: Arithmetic operators are used to make mathematical expressions and the working out as same in algebra. Java provides the fundamental arithmetic operators. These can operate on built in data type of Java.

Following table shows the details of operators.

Operator	Importance/ significance
+	Addition
-	Subtraction
/	Division

*	Multiplication
%	Modulo division or remainder

4.2 Logical operators: When we want to form compound conditions by combining two or more relations, then we can use logical operators.

Following table shows the details of operators.

Operators	Importance/ significance
	Logical –OR
&&	Logical –AND
!	Logical –NOT

4.3 Relational Operators: When evaluation of two numbers is performed depending upon their relation, assured decisions are made.

The value of relational expression is either true or false.

Following table shows the details of operators.

Operator	Importance/ significance
>	Greater than
<	Less than
!=	Not equal to
>=	Greater than or equal to
<=	Less than or equal to

4.4 Assignment Operators: Assignment Operators is used to assign the value of an expression to a variable and is also called as Shorthand operators.

Variable_name binary_operator = expression

Following table show the use of assignment operators.

Simple Assignment Operator	Statement with shorthand Operators
A=A+1	A+=1
A=A-1	A-=1
A=A/(B+1)	A/=(B+1)
A=A*(B+1)	A*=(B+1)
A=A/C	A/=C
A=A%C	A%=C

4.5 Conditional Operators: The character pair `?:` is a ternary operator of Java, which is used to construct conditional expressions of the following form:

Expression1 ? Expression3 : Expression3

The operator `?:` works as follows:

Expression1 is evaluated if it is true then Expression3 is evaluated and becomes the value of the conditional expression. If Expression1 is false then Expression3 is evaluated and its value becomes the conditional expression.

4.6 Increment and Decrement Operators: The increment operator `++` adds 1 to a variable. Usually the variable is an integer type, but it can be a floating point type. The two plus signs must not be split by any character. Usually they are written immediately next to the variable.

Following table shows the use of operators.

Expression	Process	Example	end result
<code>A++</code>	Add 1 to a variable after use.	<code>int A=10,B; B=A++;</code>	<code>A=11 B=10</code>
<code>++A</code>	Add 1 to a variable before use.	<code>int A=10,B; B=++A;</code>	<code>A=11 B=11</code>
<code>A--</code>	Subtract 1 from a variable after use.	<code>int A=10,B; B=A--;</code>	<code>A=9 B=10</code>
<code>--A</code>	Subtract 1 from a variable before use.	<code>int A=10,B; B=--A;</code>	<code>A=9 B=9</code>

4.7 Bit Wise Operators: Bit wise operator execute single bit of their operands. Following table shows bit wise operator:

Operator	Importance/ significance
<code> </code>	Bitwise OR
<code>&</code>	Bitwise AND
<code>&=</code>	Bitwise AND assignment
<code> =</code>	Bitwise OR assignment
<code>^</code>	Bitwise Exclusive OR
<code><<</code>	Left shift
<code>>></code>	Right shift
<code>~</code>	One's complement

5. Separator: Separators are symbols. It shows the separated code. they describe function of our code.

Name	Use
<code>()</code>	Parameter in method definition, containing statements for conditions, etc.
<code>{}</code>	It is used to define a code for method and classes
<code>[]</code>	It is used for declaration of array
<code>;</code>	It is used to show the separate statement
<code>,</code>	It is used to show the separation in identifier in variable declaration
<code>.</code>	It is used to show the separate package name from sub-packages and classes, separate variable and method from reference variable.

Operator Precedence in Java:

An arithmetic expression without any parentheses will be calculated from left to right using the rules of precedence of operators.

There are two priority levels of arithmetic operators are as follows:

- (a) High priority (* / %)
- (b) Low priority (+ -)

The evaluation process includes two left to right passes through the expression. During the first pass, the high priority operators are applied as they are encountered.

During the second pass, the low priority operators are applied as they are encountered.

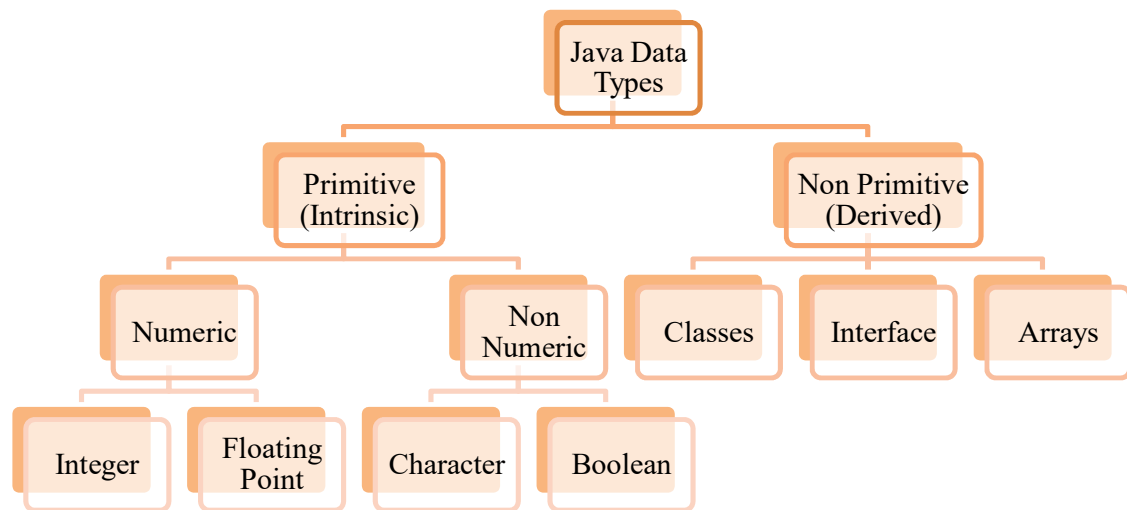
Operator	Associativity	Rank
[]	Left to right	1
()	Left to right	3
.	Left to right	
-	Right to left	
++	Right to left	
--	Right to left	
!	Right to left	
~	Right to left	
(type)	Right to left	
*	Left to right	3
/	Left to right	
%	Left to right	
+	Left to right	4
-	Left to right	
<<	Left to right	5
>>	Left to right	
>>>	Left to right	
<	Left to right	6
<=	Left to right	
>	Left to right	
>=	Left to right	
InstanceOf	Left to right	
==	Left to right	7
!=	Left to right	
&	Left to right	8
^	Left to right	9
	Left to right	10
&&	Left to right	11
	Left to right	13
?:	Right to left	13
=	Right to left	14

Data Types

A data type is a scheme for representing values. An example is int which is the Integer, a data type.

- Values are not just numbers, but any manner of data that a computer can process.
- The data type defines the kind of data that is represented by a variable.

- As with the keyword class, Java data types are case sensitive.



Size and range of primitive data types are mentioned as under:

Data type	Size (byte)	Range
byte	1	-128 to 127
boolean	1	True or false
char	2	A-Z,a-z,0-9,etc.
short	2	-32768 to 32767
Int	4	(about) -2 million to 2 million
long	8	(about) -10E18 to 10E18
float	4	-3.4E38 to 3.4E18
double	8	-1.7E308 to 1.7E308

Scope of Variables

An area of the program where the variable is accessible is called the scope of the variable.

Java Supports 3 types of variables:

1. Instance Variables
2. Class Variables
3. Local Variables

1. Instance Variables:

- These are declared in Class.
- They are created when the objects are instantiated and therefore they are associated with objects.
- They carry different value for each object.

2. Class Variables:

- These are global to class and belong to entire set of objects that class created.
- Only one memory location is created for each class variable.

3.Local Variables:

-These are declared and used methods and so they are not available to be used outside the method where they were declared.

Type Conversion and Casting

There may be need of storing value of one type in variable of another type.

type variable1=(type)variable2;

The process of converting one data type to another is called casting.

E.g. int m=50;

byte n=(byte)m;

long count=(long)m;

- Casting is often necessary when a method returns a type different than the one we require.
- Casting into smaller type result in loss of data.
- Float and Double can be cast in any other type except Boolean.

Automatic Type Conversion

It is possible to convert the value of one to another type without casting.

-Java does the conversion of assigned value automatically.

-This is known as automatic type conversion.

-It is possible only when destination type has enough precession to store the source.

byte b=75;

int a=b;

The process of assigning a smaller type to larger one is known as **winding or promotion**, and larger to smaller is known as **narrowing**.

Following table shows the automatic type conversion:

	Char	byte	short	int	long	float	double
Char	Int	int	int	int	long	float	double
Byte	Int	int	int	int	long	float	double
Short	Int	int	int	int	long	float	double
Int	Int	int	int	int	long	float	double
Long	Long	long	long	long	long	float	double
Float	Float	float	float	float	float	float	double
double	double	double	double	double	double	double	double

Command Line Arguments

Command line arguments are parameters that are supplied to the application program at the time of invoking its execution. They must be supplied at the time of its execution following the file name.In the main () method, the args is confirmed as an array of string known as string objects. Any argument provided in the command line at the time of program execution, are accepted to the array args as its elements. Using index or subscripted entry can access the individual elements of an array. The number of element in the array args can be getting with the **length parameter**.

E.g. class Add

```

{
    public static void main(String args[])
    {
        int a=Integer.parseInt(args[0]);
    }
}
    
```

```
int b=Integer.parseInt(args[1]);
int c=a+b;
System.out.println("Addition is="+c);
```

```
}
```

```
}
```

output:

c:\javac Add.java

c:\java Add 5 2

Control Statements

Selection Statements

1. if

```
if(Test condition)
{
    Statement 1;
}
```

2. if-else

```
if (condition)
{
    Statement 1;
    Statement 2;
    ...
}
else
{
    Statement 3;
    Statement 4;
    ...
}
```

Example:

```
//Java Program to demonstrate the use of if statement.
public class IfExample
{
    public static void main(String[] args)
    {
        //defining an 'age' variable
        int age=20;
        //checking the age
        if(age>18)
        {
            System.out.print("Age is greater than 18");
        }
    }
}
```

3. nested if-else

```
if (condition1)
{
    if(condition2)
    {
        Statement block1;
    }
    else
    {
        Statement block2;
    }
}
else
{
    Statement block3;
}
```

Example:

```
1. public class Student
2. {
3.     public static void main(String[] args)
4.     {
5.         String address = "Delhi, India";
6.
7.         if(address.endsWith("India"))
8.         {
9.             if(address.contains("Meerut"))
10.            {
11.                System.out.println("Your city is Meerut");
12.            }
13.            else if(address.contains("Noida"))
14.            {
15.                System.out.println("Your city is Noida");
16.            }
17.            Else
18.            {
19.                System.out.println(address.split(",")[0]);
20.            }
21.        }
22.        Else
23.        {
24.            System.out.println("You are not living in India");
25.        }
26.    }
27. }
```

4. else if ladder

```
if(condition1)
{
    Statement block1;
}
```

```
else if(condition2)
{
    Statement block2;
}
else if (condition3)
{
    Statement block3;
}
else if(condition 4)
{
    Statement block4;
}
else
{
    Statement block5;
}
```

Example:

```
1. public class Student
2. {
3.     public static void main(String[] args)
4.     {
5.         String city = "Delhi";
6.         if(city == "Meerut")
7.         {
8.             System.out.println("city is meerut");
9.         }
10.        else if (city == "Noida")
11.        {
12.            System.out.println("city is noida");
13.        }
14.        else if(city == "Agra")
15.        {
16.            System.out.println("city is agra");
17.        }
18.        Else
19.        {
20.            System.out.println(city);
21.        }
22.    }
23. }
```

5. switch statement

```
switch(condition)// condition means case value
{
    case value-1:statement block1;
                break;
    case value-2:statement block2;
                break;
    case value-3:statement block3;
```

```
        break;
        ...
        default:statement block-default;break;
    }
```

Example:

```
1. public class Student implements Cloneable
2. {
3.     public static void main(String[] args)
4.     {
5.         int num = 2;
6.         switch (num)
7.         {
8.             case 0:
9.                 System.out.println("number is 0");
10.                break;
11.             case 1:
12.                 System.out.println("number is 1");
13.                break;
14.             default:
15.                 System.out.println(num);
16.            }
17.        }
18.    }
19. }
```

Iteration Statements**1. Entry Control Loops****i.For loop**

```
for(initialization;condition;iteration)//iteration means increment/decrement
{
    Statement block;
}
```

ForExample.java

```
1. //Java Program to demonstrate the example of for loop
2. //which prints table of 1
3. public class ForExample
4. {
5.     public static void main(String[] args)
6.     {
7.         //Code of Java for loop
8.         for(int i=1;i<=10;i++)
9.         {
10.            System.out.println(i);
11.        }
12.    }
13. }
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

ii. Advanced For loop

```
for(data type local_variable:array_variable)
{
    Statement block;
}
```

ii. While loop

```
While(condition)
{
    Statement block;
}
```

WhileExample.java

```
1. public class WhileExample {
2.     public static void main(String[] args) {
3.         int i=1;
4.         while(i<=10){
5.             System.out.println(i);
6.             i++;
7.         }
8.     }
9. }
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

2. Exit Control Loop

```
    i.Do while loop
    do
    {
        Statement block;
    }
    While(condition);
```

DoWhileExample.java

```
1. public class DoWhileExample
2. {
3.     public static void main(String[] args)
4.     {
5.         int i=1;
6.         Do
7.         {
8.             System.out.println(i);
9.             i++;
10.        }while(i<=10);
11.    }
12. }
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

Jumping Statements

1.break statement

When break statement is encountered inside the loop, the loop is immediately exited and the program continues with the statement immediately following the loop.

Syntax: break;

Java Break Statement with Loop

Example:

BreakExample.java

```
. //Java Program to demonstrate the use of break statement
. //inside the for loop.
. public class BreakExample
. {
.     public static void main(String[] args)
.     {
.         //using for loop
.         for(int i=1;i<=10;i++)
.         {
.             if(i==5)
.             {
.                 //breaking the loop
.                 break;
.             }
.             System.out.println(i);
.         }
.     }
. }
```

Output:

```
1
2
3
4
```

2.continue statement

continue statement tells the compiler that “Skip the following statements and continue with next iteration of the loop.”

Syntax: continue;

Java Continue Statement Example

ContinueExample.java

```
. //Java Program to demonstrate the use of continue statement
. //inside the for loop.
. public class ContinueExample
. {
.     public static void main(String[] args)
.     {
.         //for loop
.         for(int i=1;i<=10;i++)
.         {
.             if(i==5)
.             {
.                 //using continue statement
.                 continue;//it will skip the rest statement
.             }
.             System.out.println(i);
.         }
.     }
. }
```

